

Functional Programming

Lab 01: A Simple CSS Parser

V. Markos

Mediterranean College

Spring, 2024 – 2025, Week 22

Abstract

This mini lab series aims to build a simple CSS parser, aiming mostly to get some hands on experience with parsing stuff in a Functional way in Python. In this first lab we aim to get a (supposedly) valid CSS string as input and return a proper Abstract Syntax Tree (AST) as output.

1 Getting Started

You can find some useful resources in the `./source` directory for this lab. Namely:

- `main.py`, which serves as the main source of execution for our project.
- `Token`, which implements the `TokenType` class and any required support classes.
- `Parser`, which implements the actual scanner (lexer) and parser for this lab.

None of the above files contains a full implementation of the requested functionalities for this lab, however they all provide some (minor or major) hints towards that. The rest is up to you to fill-in and improvise. You can always feel free to ignore those files and start from scratch on your own.

Also, as a note for all labs, if you do not feel comfortable enough with CSS and its syntax, you might want to have a look at the following resources:

- MDN CSS Tutorial: <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- CSS Tricks: <https://css-tricks.com/>.

You can also use the source file `main.py` as the single entry point to your parser (which you can modify accordingly):

```

1 from Parser import parse, scan_tokens
2 from os import path
3
4 # Alter this to fit your project's structure
5
6 def main():
7     response = input("Press 'q' to exit or any key to start!")
8     while response != "q":
9         filepath = input("Enter CSS file path to parse: ")
10        with open(filepath) as css_file:
11            css_source = css_file.read()
12            tokens = scan_tokens(css_source)
13            ast = parse(tokens)
14            print("AST:")
15            print(ast) # You might need a function to stringify your
                        AST

```

A general comment for all labs: stay functional^a! That is, avoid mutations, side effects and all those impurities as much as possible. To do so, consider using comprehensions instead of loops, or, even better, iterators and generators wherever possible. Moreover, instead of mutating existing objects, build your functions in a way that they do not mutate their inputs but, instead, return copies of them, with any additions needed made on those copies.

^aThere are so many ways to interpret this, but, please, do so in the context of the module. Performing key human functionalities is not within the scope of this lab.

2 Tokenisation

The first step in scanning / lexing is to properly tokenise the provided source code. To do so, we have to, as we formally say, split the source code into *lexemes*, i.e., the quanta of meaning in a source file. As you might already suspect, all lexemes are not born equal. There are some characters that have a very specific meaning in a language, e.g., parentheses, brackets, reserved keywords, while some others can have any meaning from a particular list of choices, e.g., variable names, numerical constants, etc. One way to distinguish between all those is to use different token types for tokens that fall in some of the above categories. With that in mind, here comes your first exercise:

Exercise 2.1. Think of what token types you might need for a simple CSS parser. You can brainstorm, look around the web, open up the source code of a webpage and look how CSS looks like, or whatever, to draw some inspiration. Having done so, edit `Token.py` (or your own corresponding source file) and add any token types you see fit in the `TokenType` enumeration class. Some classical token types have been already written for you:

```

1 from dataclasses import dataclass
2 from enum import Enum

```

```

3
4 # Enum is not just a Python class, since you can do many things
   with it.
5 # Also, dataclasses are roughly like Haskell's ADTs, so, no actual
   member methods etc.
6
7 @dataclass
8 class TokenType(Enum):
9     LEFT_PAREN = "LEFT_PAREN"
10    RIGHT_PAREN = "RIGHT_PAREN"
11    CLASS = "CLASS"
12    PROPERTY_NAME = "PROPERTY_NAME"
13    # Add more CSS token types here...

```

Hint: If all this feels overwhelming, focus on simple CSS class / tag declarations, ignoring potential nesting and or more complex syntax, such as media queries etc.

Given a few token types, we now have to actually parse them. So, here comes your next exercise:

Exercise 2.2. Implement the `Parser.scan_tokens()` method, as found in `Parser.py` with the aim of properly scanning for simple CSS scripts. This function should get a piece of CSS source code and return a list of the corresponding tokens or raise appropriate exceptions — which you will have to handle appropriately. Some things you might want to take into account for your implementation might include:

- You better implement a few helper functions to help you recognise operators, digits, floats, integers, parenthesised expressions and anything else you see fit, depending on how deep a dive you want to take into CSS.
- Reserve parentheses / brackets and their balancing for last, since this might be a bit tricky.
- You will need to keep track of where you are standing in the provided source, in terms of line and column (i.e., character within that line), so, feel free to add them as class attributes or local variables in your functions. You will also need this piece of information for error logging.
- You might want to utilise regular expressions to facilitate parsing, however, note that in subsequent labs you will most probably need to drop this approach, as it makes several computations and pattern matching quite heavy in the case of more complex expressions. So, to save yourselves from some future work, avoid depending that much on regex.

3 Grammars

Having a list of tokens, it actually remains to make sense of them. To that end, we have to craft a Context-Free Grammar (CFG) that can capture the basic CSS constructs we would like to recognise. So:

Exercise 3.1. Consider how maths expressions are structured and how any relevant tokens are related with each other. Then, craft a CFG that captures all those syntactic rules you have spotted. Use the format presented in slides or any other format you find convenient.

Hint: Your grammar should not be too long, given the simplicity of the language at hand.

Exercise 3.2. Create an `AST.py` Python file and define an `AST` (data)class in there. Then, define any functions needed to properly parse a list of tokens into an AST. Remember to stay functional and do not make any mutations or generate side effects!

Hint: Recall your last semester's assignment!